

ZAP! Programming Language - Getting Started Guide

The author of this software stands in solidarity with  Ukraine. We believe in a world where international borders are respected and human rights are upheld. We encourage all users of this software to contribute to humanitarian efforts in  Ukraine.

Beginner's Guide to Programming in ZAP!

Version: 1.0

Date: January 2026

Table of Contents

1. [What is ZAP!?](#)
 2. [Installation](#)
 3. [Your First Program](#)
 4. [Basic Concepts](#)
 5. [Working with Variables](#)
 6. [Making Decisions](#)
 7. [Loops](#)
 8. [Functions and Procedures](#)
 9. [Troubleshooting](#)
 10. [Next Steps](#)
-

What is ZAP!?

ZAP! is a programming language designed for **8-bit systems**, particularly the **Atari 8-bit computer** family. It's modern yet close to the metal, giving programmers both high-level constructs and low-level control.

Why ZAP!?

- **Simple Syntax** - Easy to learn for beginners
- **Powerful Optimizations** - Compiler generates efficient 6502 assembly
- **Hardware Access** - Direct access to memory-mapped I/O and hardware registers
- **Retro Computing** - Perfect for Atari 8-bit development and homebrew games
- **Educational** - Great for learning assembly language concepts

Who Should Learn ZAP!?

- Game developers targeting Atari 8-bit systems
 - Hobbyists interested in retro computing
 - Students learning low-level programming
 - Anyone interested in 6502 assembly without writing assembly directly
-

Installation

Prerequisites

1. **ZAP!** - For compiling ZAP programs
 - Download from: <https://github.com/Dushino/ZAP-compiler/releases>
2. **CC65 Toolchain** - For assembling/linking ZAP! generated assembler source files
 - Includes ca65 (assembler) and ld65 (linker)
 - Download from: <https://cc65.github.io/>
3. **IDE** (optional) - For editing ZAP source code VS Code and Antigravity are supported
 - Download from: <https://code.visualstudio.com/>
 - Download from: <https://antigravity.google/>
4. **Python 3.x** (optional) - For ZAP! compiler development
 - Download from: <https://www.python.org/downloads/>

Setting Up ZAP!

Just be sure zapc is in your PATH.

Installing CC65

For Atari-specific development:

Ubuntu/Debian:

```
sudo apt-get install cc65
```

macOS (with Homebrew):

```
brew install cc65
```

Windows: Download from: <https://cc65.github.io/>

Building the Compiler from Source (Optional)

If you want to build the ZAP! compiler executable from source (instead of using a pre-built release), you need Python 3.x and PyInstaller.

1. Install PyInstaller:

```
pip install pyinstaller
```

2. Run the build script:

Windows:

```
make_dist.bat
```

Linux/macOS:

```
./make_dist.sh
```

This creates a standalone **zabc** executable in the **dist/** directory. Copy it to a location in your PATH.

Installing VS Code Extension (Optional)

Syntax highlighting and snippets are available for VS Code / Antigravity.

Windows:

```
IDE_Integration\install_vscode_extension.bat
```

Linux/macOS:

```
./IDE_Integration/install_vscode_extension.sh
```

Note: After installation, you must reload the editor for the changes to take effect.

Your First Program

Let's start with the simplest possible ZAP program:

Hello World

Create a file named **hello.zap**:

```
proc main()  
    ; Your code goes here  
end
```

This is the **minimal** ZAP program. Every program needs:

1. A **main** procedure
2. At least **proc main()** and **end**

Compile It

```
zapc --cpu 6502 -O1 hello.zap -o hello.s
```

This generates `hello.s` (6502 assembly code).

A More Interesting Example

Let's declare a variable:

```
byte message_code

proc main()
    message_code = 65    ; ASCII for 'A'
end
```

Save as `first.zap` and compile:

```
zapc --cpu 6502 -O1 first.zap -o first.s
```

Assemble and Link

```
ca65 first.s -o first.o
ld65 first.o -o first.prg
```

Basic Concepts

Procedures and Functions

Procedures do things but don't return values:

```
proc greet()
    ; Do something
end
```

Functions return values:

```
func byte add(byte a, byte b)
    return a + b
end
```

Variables

Variables store data:

```
byte health = 100      ; 8-bit number (0-255)
word score = 0         ; 16-bit number (0-65535)
byte ^ptr              ; Pointer (advanced)
```

Control Flow

Programs make decisions and repeat:

```
if condition
    ; Do this
else
    ; Do that
end

while condition
    ; Repeat this
end

for i = 0 to 10
    ; Repeat 10 times (0 through 9)
end
```

Working with Variables

Declaring Variables

```
byte score              ; Uninitialized
byte level = 1          ; With initial value
byte health = 100       ; With assignment
```

Variable Types

Type	Range	Purpose
byte	0-255	Small numbers, characters, ASCII values
word	0-65535	Larger numbers, addresses
long	0-4294967295	32-bit counters, large values, timestamps

```
byte health = 100      ; Character health (0-255)
word high_score = 5000 ; High score (can be > 255)
long population = 65536 ; 32-bit value (cannot fit in word)
```

Using Variables

```
byte x = 10
byte y = 20
byte sum = x + y      ; sum = 30

x = x + 1              ; x = 11
y = y - 5              ; y = 15
```

⚠ Initialize Before Use

Important: The ZAP! compiler requires that all variables are initialized before they are used. This prevents bugs from uninitialized memory:

```
proc safe_example()
    byte x = 0          ; Initialize first
    byte y = x + 10     ; OK: x has a value
end
```

This will **fail to compile**:

```
proc unsafe_example()
    byte x              ; Not initialized!
    byte y = x + 10     ; ERROR: Use of uninitialized variable 'x'
end
```

The compiler checks all code paths to ensure variables are initialized before use. This is like Rust's safety features, but for 6502 assembly! See [Uninitialized Variable Detection](#) for full details.

Global vs Local Variables

Global (available everywhere):

```
byte game_over = 0

proc main()
    game_over = 1      ; Accessible
end
```

Local (only in procedure):

```
proc main()  
    byte local_var = 10 ; Only here  
end  
  
proc other()  
    ; local_var not accessible here  
end
```

Static Local Variables

Static variables are local variables that persist their value between procedure calls. Unlike regular local variables (which are re-initialized on each procedure entry), static variables are initialized only once at program start, and retain their value across calls.

Syntax:

```
proc counter()  
    static byte count = 0 ; Initialized once at program start  
    count = count + 1  
end  
  
proc main()  
    counter() ; count becomes 1  
    counter() ; count becomes 2  
    counter() ; count becomes 3  
end
```

Rules for STATIC variables:

- Can only be used on **local variables** (inside procedures/functions)
- Cannot be combined with **CONST**
- **Must have an initializer** (the initial value)
- The initializer is executed once at program startup, after global variables are initialized
- The variable retains its value between procedure calls

Use cases:

- Call counters
- State machines
- Configuration flags
- Resource pools

Example: Simple Call Counter

```
func byte get_next_id()
    static byte next_id = 1
    byte result = next_id
    next_id = next_id + 1
    return result
end
```

Example: Score Tracker

```
byte current_score = 0
byte high_score = 0

proc add_points(byte points)
    current_score = current_score + points

    if current_score > high_score
        high_score = current_score
    end
end

proc main()
    add_points(10)
    add_points(25)
    add_points(15)
    ; high_score is now 50
end
```

Making Decisions

The if Statement

```
proc check_age(byte age)
    if age >= 18
        ; You're an adult
    end
end
```

if-else

```
proc check_age(byte age)
    if age >= 18
        ; Adult code
    else
        ; Minor code
    end
end
```



```
end  
end
```

if-elseif-else

When you need to check multiple conditions in sequence, use **elseif** to avoid deeply nested blocks:

```
proc check_level(byte level)  
  if level < 3  
    ; Beginner  
  elseif level < 7  
    ; Intermediate  
  elseif level < 10  
    ; Advanced  
  else  
    ; Expert  
  end  
end
```

elseif is equivalent to writing **else** followed by a nested **if**, but without needing an extra **end** for each branch.

Comparison Operators

Operator	Meaning	Example
==	Equal to	x == 5
!=	Not equal to	x != 0
<	Less than	x < 100
>	Greater than	x > 0
<=	Less or equal	x <= 255
>=	Greater or equal	x >= 1

```
proc game_logic(byte level)  
  if level == 1  
    ; Easy level  
  end  
  
  if level < 5  
    ; Early levels  
  end  
  
  if level > 9  
    ; Hard levels  
  end
```

```
    end  
end
```

Combining Conditions

AND (&&) - Both must be true:

```
proc enter_dungeon(byte level, byte health)  
    if level >= 5 && health > 50  
        ; Can enter  
    end  
end
```

OR (||) - Either can be true:

```
proc is_ready(byte stamina, byte magic)  
    if stamina > 20 || magic > 20  
        ; Ready to fight  
    end  
end
```

NOT (!) - Reverse the condition:

```
proc check_not_dead(byte health)  
    if !(health == 0)  
        ; Still alive!  
    end  
end
```

Nested Conditions

```
proc complex_logic(byte a, byte b)  
    if a > 10  
        if b < 5  
            ; a > 10 AND b < 5  
        end  
    end  
end
```

Example: Game State Logic

```
byte game_state = 0    ; 0=menu, 1=playing, 2=paused, 3=over

proc update_game()
  if game_state == 0
    ; Show menu
  elseif game_state == 1
    ; Update gameplay
  elseif game_state == 2
    ; Show pause screen
  elseif game_state == 3
    ; Show game over
  end
end
```

Loops

The while Loop

Repeat while a condition is true:

```
byte counter = 0

proc count_to_ten()
  while counter < 10
    counter = counter + 1
  end
end
```

The repeat-until Loop

Repeat at least once, stopping when the condition becomes true:

```
byte counter = 0

proc count_to_five()
  repeat
    counter = counter + 1
  until counter == 5
end
```

Infinite Loop:

```
proc main_game_loop()
  while 1      ; Always true - infinite loop
    update_game()
```

```
        draw_screen()
        check_input()
    end
end
```

Breaking Out of Loops

Use **break** to exit early:

```
byte found = 0
byte i

proc search_for_value(byte target_value)
    for i = 0 to 255
        if memory_at(i) == target_value
            found = 1
            break          ; Exit loop
        end
    end
end
end
```

The for Loop

Repeat a set number of times:

Note: The **to** bound is exclusive (C-like). For example, **for i = 0 to 9** runs 9 iterations (0..8), and **for i = 0 to 1** runs once. If you need to include a specific last value, set **to** one step past it (e.g., **for i = 0 to 10** includes 9 with step 1, or **for i = 0 to 110 step 10** includes 100).

```
byte i

proc count_ten()
    for i = 0 to 9
        ; Executes 9 times (i = 0, 1, 2, ... 8)
    end
end
```

With Step:

```
byte i

proc count_by_tens()
  for i = 0 to 100 step 10
    ; i = 0, 10, 20, 30, ... 90
  end
end
```

Backwards (using while):

```
byte i

proc count_down()
  i = 10
  while i > 0
    ; i = 10, 9, 8, ... 1
    i = i - 1
  end
end
```

Note: `for` loops only support positive step values. Use `while` for counting down.

Example: Initialize Screen

```
byte i
byte x, y

proc clear_screen()
  ; Clear using a loop
  for i = 0 to 255
    screen_memory[i] = 0
  end
end

proc fill_pattern()
  ; Fill with checkerboard pattern
  for x = 0 to 39
    for y = 0 to 24
      screen_memory[y * 40 + x] = (x + y) & 1
    end
  end
end
```

Functions and Procedures

Procedures (No Return Value)

```
proc say_hello()  
    ; Just do something  
end  
  
proc add_one_to_variable(byte var)  
    var = var + 1  
end  
  
proc main()  
    say_hello()  
end
```

Note: Changes to parameters don't affect the original:

```
byte x = 10  
  
proc increment(byte val)  
    val = val + 1  
end  
  
proc main()  
    increment(x)  
    ; x is still 10 (procedure got a copy)  
end
```

Deep copy note: Parameters are passed by value. Scalars are copied, structs are copied byte-for-byte, and pointers copy only the address (not the data). To modify caller data, pass a pointer.

Functions (Return Values)

```
func byte double_value(byte input)  
    return input * 2  
end  
  
func word combine_bytes(byte low, byte high)  
    return low + (high * 256)  
end  
  
proc main()  
    byte result = double_value(21) ; result = 42  
    word address = combine_bytes(0, $40) ; address = $4000  
end
```

Parameters

```
; No parameters
proc greet()
end

; One parameter
proc greet_person(byte name)
end

; Multiple parameters
proc calculate(byte a, byte b, byte c)
end

; With different types
func byte compute(byte x, word y)
    return x + (y & 255)
end
```

Using Return to Exit Early

```
proc validate_input(byte input)
    if input == 0
        return      ; Exit procedure early
    end

    ; Rest of validation here
end
```

Example: Utility Functions

```
func byte min(byte a, byte b)
    if a < b
        return a
    end
    return b
end

func byte max(byte a, byte b)
    if a > b
        return a
    end
    return b
end

func byte abs_diff(byte a, byte b)
    if a > b
        return a - b
    end
    return b - a
end
```

```
end

proc main()
    byte smallest = min(10, 20)      ; 10
    byte largest = max(10, 20)      ; 20
    byte distance = abs_diff(5, 15) ; 10
end
```

Arrays

Declaring Arrays

```
byte data[10]      ; Array of 10 bytes, uninitialized
byte data[] = {1,2,3} ; Size determined by initializer
```

Accessing Array Elements

```
byte arr[5] = {10, 20, 30, 40, 50}

proc main()
    byte first = arr[0]      ; 10
    byte third = arr[2]      ; 30
    arr[1] = 99              ; Change second element
end
```

Arrays in Loops

```
byte data[256]
byte i

proc initialize_array()
    for i = 0 to 255
        data[i] = i
    end
end

proc sum_array()
    byte sum = 0
    for i = 0 to 255
        sum = sum + data[i]
    end
end
```

Strings

Strings are arrays of characters:

```
byte greeting[] = "Hello"

proc main()
  byte first_char = greeting[0] ; 'H' = 72
  byte second = greeting[1]      ; 'e' = 101
end
```

Example: High Score Table

```
byte score1 = 100
byte score2 = 85
byte score3 = 60
byte scores[] = {100, 85, 60}

proc add_score(byte new_score)
  if new_score > scores[0]
    scores[2] = scores[1]
    scores[1] = scores[0]
    scores[0] = new_score
  end
end
```

Troubleshooting

Program Won't Compile

Error: "Undefined variable"

```
proc main()
  x = 5 ; ERROR: x not declared
end
```

Fix: Declare the variable first:

```
byte x
proc main()
  x = 5 ; OK
end
```

Error: "Duplicate variable"

```
byte x = 5
byte x = 10 ; ERROR: x already exists
```

Fix: Use different names or different scopes:

```
byte global_x = 5
proc test()
    byte local_x = 10 ; OK: different scope
end
```

Logic Errors

Loop doesn't execute:

```
byte i
for i = 10 to 5 ; Won't run - start > end
    ; Never runs
end
```

Need to count down? Use while:

```
; for loops only go up, use while to count down:
i = 10
while i > 5
    ; Process i = 10, 9, 8, 7, 6
    i = i - 1
end
```

Variable changes don't persist:

```
proc modify(byte x)
    x = 0 ; Changes local copy only
end

byte my_var = 10
modify(my_var) ; my_var still 10!
```

Fix: Use return value or global:

```
proc modify()
    global_x = 0 ; Changes global
```

```
end
```

Performance Issues

Program too slow?

- Avoid nested loops in time-critical sections
- Use byte arithmetic instead of word when possible
- Consider using arrays for lookup tables instead of calculations

Running out of memory?

- Reduce array sizes
 - Check for memory-mapped I/O conflicts
 - Use fixed addresses for arrays to optimize placement
-

Next Steps

Learn More

1. **Read the [ZAP! Language Reference](#)** - Complete language documentation
2. **Study example programs** - Look in [tests/pass/](#) directory
3. **Explore advanced features** - Pointers, inline assembly, optimization

Practice Exercises

1. Counter Program

- Create a counter that increments from 0 to 255
- Display the counter value

2. Array Sum

- Create an array of 10 numbers
- Calculate and display their sum

3. Game Loop

- Create a simple game loop that updates game state
- Add player input handling

4. Lookup Table

- Create a sine or cosine lookup table
- Use it to animate something on screen

Build a Project

Create a simple game or utility:

- Ball bouncing simulator

- Menu system
- Sprite controller
- Score tracker
- Music note sequencer

Join the Community

- Report bugs or ask questions at: <https://github.com/Dushino/ZAP-compiler>
- Read compiler documentation for advanced optimization options
- Explore Atari 8-bit programming resources

Key Takeaways

- ✓ **Variables** store data
- ✓ **Procedures** execute code
- ✓ **Functions** return values
- ✓ **if/else/end** make decisions
- ✓ **while/for/repeat-until** loops repeat code
- ✓ **Arrays** store multiple values

Your First Real Program Template

```
; Global variables
byte state = 0
byte counter = 0

; Initialize everything
proc initialize()
    state = 1
    counter = 0
end

; Main game/app logic
proc update()
    counter = counter + 1

    if counter > 100
        state = 0
        counter = 0
    end
end

; Main entry point
proc main()
    initialize()

    while state == 1
        update()
```

```
end  
end
```

Appendix: Cheat Sheet

Variable Declaration

```
byte name           ; 0-255  
word name           ; 0-65535  
byte name = value   ; With initializer  
const byte NAME = 10 ; Constant
```

Control Flow

```
if x == 5 ... elseif x == 3 ... else ... end  
while x < 10 ... end  
for i = 0 to 9 ... end  
repeat ... until x == 10  
break  
return [value]
```

Functions & Procedures

```
proc name() ... end  
func type name(params) ... return value end  
name(args)
```

Arrays

```
byte arr[10]  
byte arr[] = {1,2,3}  
value = arr[0]  
arr[0] = value
```

Operators

```
+, -, *, /, %           ; Math  
==, !=, <, >, <=, >=    ; Compare  
&&, ||, !               ; Logic
```

Ready to start programming? Begin with the [ZAP! Language Reference](#)!